

Partie 1 — Diagnostic et Simulation des Manquants

Q 1.1 Chargement et profil initial du dataset

```
print(f"Dimensions : {data.shape}")
print(f"Types de données :\n{data.dtypes.value_counts()}")
print(f"Variables numériques : {data.select_dtypes(include=['int64', 'float64']).shape[1]}")
print(f"Variables catégorielles : {data.select_dtypes(include=['object']).shape[1]}")
```

```
Dimensions : (10127, 19)
Types de données :
int64      10
object      6
float64     3
Name: count, dtype: int64
Variables numériques : 13
Variables catégorielles : 6
```

```
# Q1.1 - Question 2 : Ratio Existing/Attrited
attrition_counts = data['Attrition_Flag'].value_counts()
attrition_ratio = attrition_counts['Existing Customer'] / attrition_counts['Attrited Customer']
print(f"Ratio Existing / Attrited : {attrition_ratio:.2f}")
print(f"Proportion Attrited : {attrition_counts['Attrited Customer']/len(data)*100:.1f}%")
```

```
Ratio Existing / Attrited : 5.22
Proportion Attrited : 16.1%
```

```
# Q1.1 - Question 3 : Variable avec le plus grand écart
ecarts = data.describe().T['max'] - data.describe().T['min']
colonne_max_ecart = ecarts.idxmax()
print(f"Variable avec le plus grand écart (max-min) : {colonne_max_ecart}")
print(f"Écart : {ecarts.max():.2f}")
```

```
Variable avec le plus grand écart (max-min) : CLIENTNUM
Écart : 120261000.00
```

Questions d'analyse

1. 13 variables numériques, 6 variables catégorielles
2. Ratio = 5,22 (16,1% de la classe minoritaire). Dataset déséquilibré → utiliser **F1-score** ou **AUC-ROC** (pas l'accuracy)
3. CLIENTNUM (identifiant, à supprimer). Parmi les vraies features : Avg_Open_To_Buy (écart 34 513) → utiliser **RobustScaler** (résiste aux outliers)

Q 1.2 Simulation de valeurs manquantes (mécanisme MCAR)

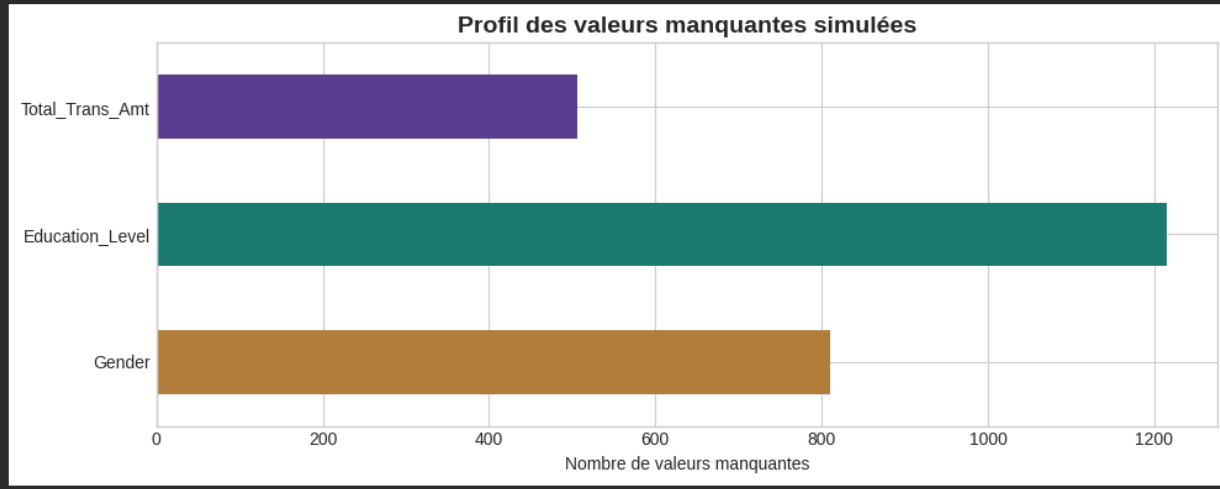
```
rng = np.random.default_rng(42)
datam = data.copy()

missing_config = {
    'Gender': 0.08,
    'Education_Level': 0.12,
    'Total_Trans_Amt': 0.05,
}

for col, rate in missing_config.items():
    n_missing = int(len(datam) * rate)
    idx = rng.choice(datam.index, size=n_missing, replace=False)
    datam.loc[idx, col] = np.nan
    print(f'{col:25s} → {n_missing} NaN ({rate*100:.0f}%)')
```

```
Simulation de valeurs manquantes (MCAR)
Gender                → 810 NaN (8%)
Education_Level       → 1215 NaN (12%)
Total_Trans_Amt       → 506 NaN (5%)
```

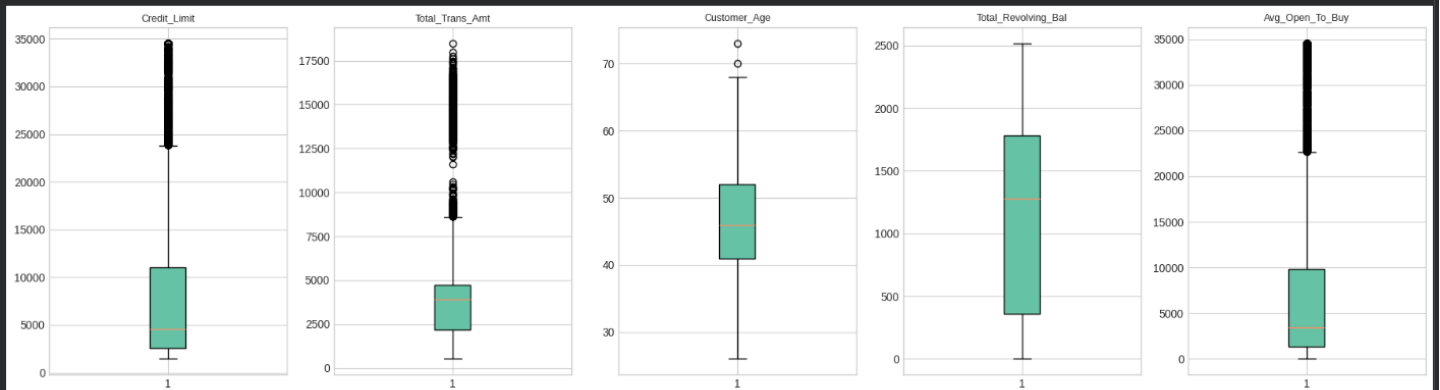
```
# Visualisation du profil des manquants
missing = datam.isnull().sum()[lambda s: s > 0]
fig, ax = plt.subplots(figsize=(10, 4))
missing.plot(kind='barh', color=['#B07D3A', '#1A7A6E', '#5B3D8F'], ax=ax)
ax.set_title('Profil des valeurs manquantes simulées', fontweight='bold', fontsize=14)
ax.set_xlabel('Nombre de valeurs manquantes')
plt.tight_layout()
plt.show()
```



4. replace=False = pas de remise → chaque indice n'est sélectionné qu'une seule fois (pas de doublons inutiles).
5. **MCAR** = manquant complètement aléatoire, indépendant de toutes les données. Le plus simple à traiter car l'échantillon reste représentatif (pas de biais).
6. **Exemple MNAR** : Credit_Limit manquante surtout pour les clients avec très faible limite (< 2000€), car ils refusent de la déclarer par honte/peur.

Q 1.3 Détection des outliers — méthode IQR

```
num_key = ['Credit_Limit', 'Total_Trans_Amt', 'Customer_Age', 'Total_Revolving_Bal', 'Avg_Open_To_Buy']
fig, axes = plt.subplots(1, 5, figsize=(18, 5))
for ax, col in zip(axes, num_key):
    ax.boxplot(data[col].dropna(), patch_artist=True)
    ax.set_title(col, fontsize=9)
plt.tight_layout(); plt.show()
```



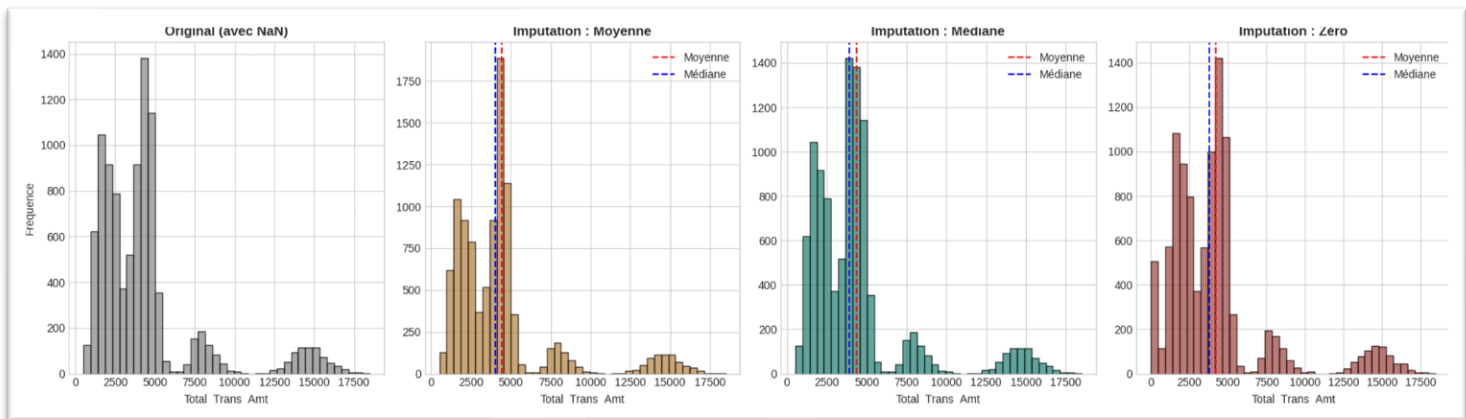
```
# Quantification IQR
print(f'Variable:30s} {N outliers':>12s} {'%':>8s}')
for col in num_key:
    q1, q3 = data[col].quantile([0.25, 0.75])
    iqr = q3 - q1
    mask = (data[col] < q1 - 1.5*iqr) | (data[col] > q3 + 1.5*iqr)
    n = mask.sum()
    print(f'{col:30s} {n:>12d} {n/len(data)*100:>7.1f}%')
```

Variable	N outliers	%
Credit_Limit	984	9.7%
Total_Trans_Amt	896	8.8%
Customer_Age	2	0.0%
Total_Revolving_Bal	0	0.0%
Avg_Open_To_Buy	963	9.5%

7. Credit_Limit (9,7%) et Avg_Open_To_Buy (9,5%) — près de 10% d'outliers
8. MinMaxScaler est très sensible aux outliers → les valeurs extrêmes écrasent les 90% de données normales dans un petit intervalle, perdant l'information
9. RobustScaler car :
 - Résistant aux outliers (basé sur médiane/IQR)
 - 10% des données sont des outliers
 - Variables financières asymétriques avec longue traîne

Partie 2 — Imputation des Valeurs Manquantes

Q 2.1 Imputation numérique — 3 stratégies comparées sur Total_Trans_Amt



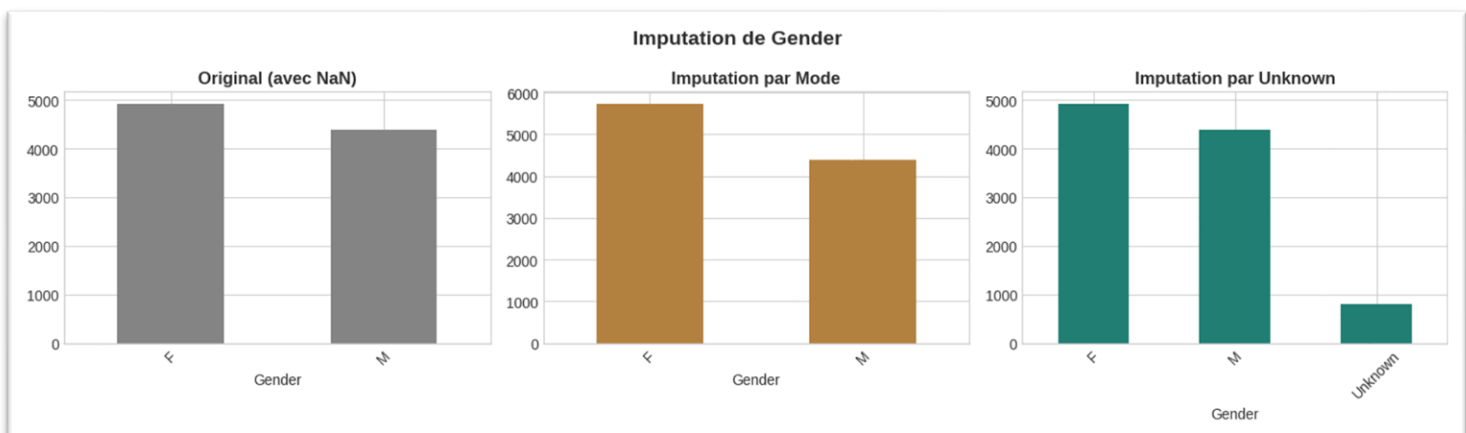
```
# Comparaison statistique
print("Comparaison des stratégies d'imputation :")
print(f"{'Stratégie':15s} {'Moyenne':>10s} {'Médiane':>10s} {'Std':>10s}")
for name, df_imp in data_imputed.items():
    s = df_imp[col]
    print(f"{'name':15s} {s.mean():>10.1f} {s.median():>10.1f} {s.std():>10.1f}")
```

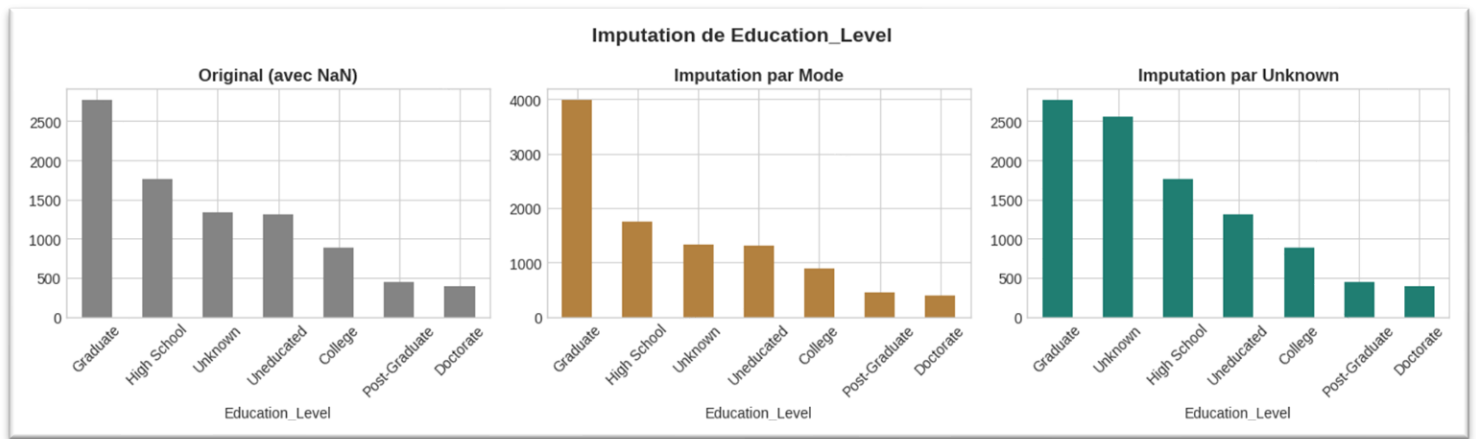
Comparaison des stratégies d'imputation :

Stratégie	Moyenne	Médiane	Std
Moyenne	4410.0	4016.0	3320.3
Médiane	4384.4	3899.0	3322.2
Zéro	4189.6	3768.0	3456.5

10. La **médiane** — préserve mieux la forme car résistante aux outliers.
11. La **médiane** — variable asymétrique avec outliers → la médiane représente mieux le client "typique" (contrairement à la moyenne tirée par les extrêmes).
12. Crée un **pic artificiel à zéro** → fausse modalité, biaise le modèle, détériore les performances. À éviter.

Q 2.2 Imputation catégorielle — Mode vs Constante 'Unknown'





13. Le **Mode** — préserve la répartition 53/47. Unknown n'a pas de sens pour le genre.

14. La constante '**Unknown**' — cohérent avec les Unknown déjà présents (fusion naturelle des non-réponses).

15. map() retourne **NaN** pour toute valeur absente du mapping → NaN non détectés → erreur ou résultats faussés.
Toujours vérifier avec assert.

Partie 3 — Encodage des Variables Catégorielles

Q 3.1 Label Encoding — variables binaires et variable cible

```
# Vérification qu'aucun NaN n'a été introduit
check_cols = ['Attrition_Flag', 'Gender', 'Marital_Status']
print(f"NaN introduits : {df_enc[check_cols].isna().sum().sum()}")
assert df_enc[check_cols].isna().sum().sum() == 0, "Des NaN ont été introduits !"

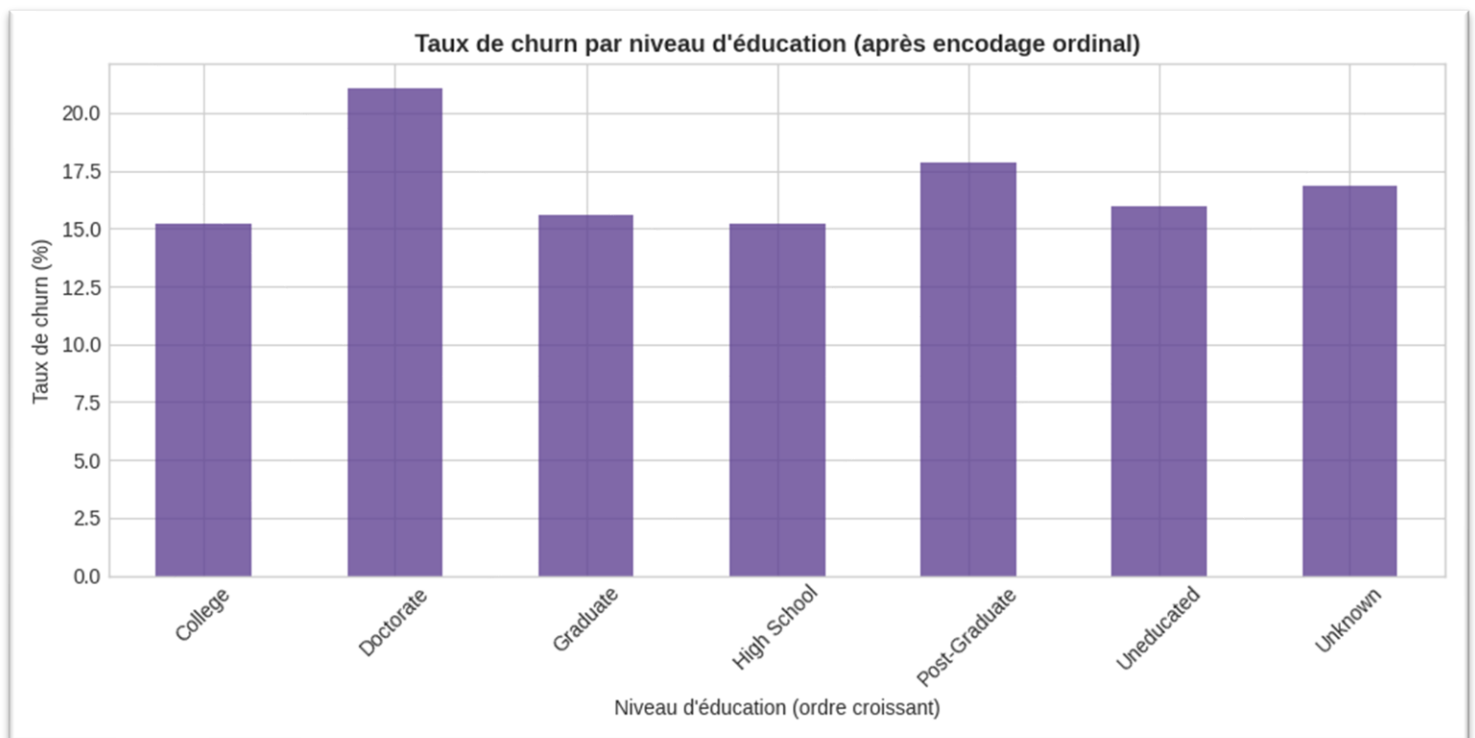
NaN introduits : 0
```

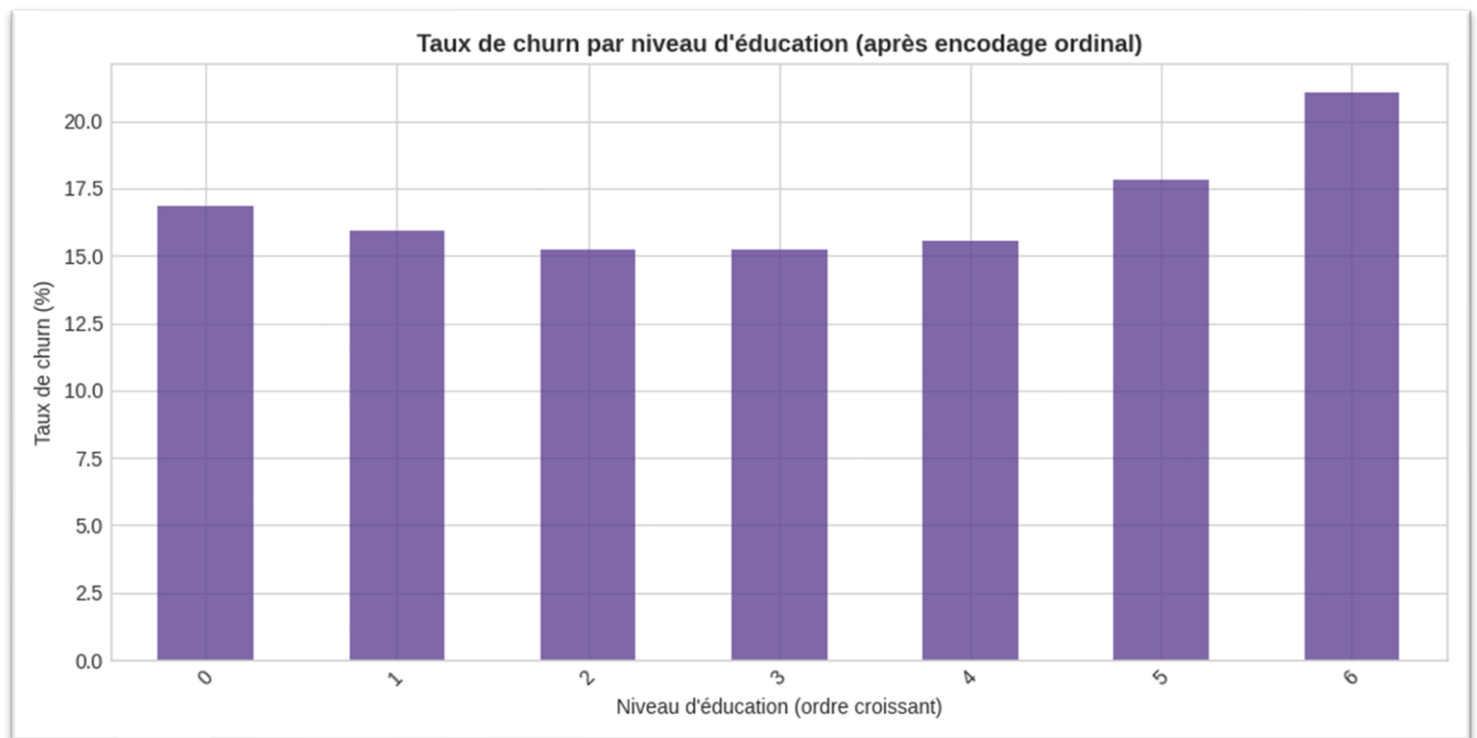
16. Attrited Customer = 1 car :

- 1 = événement (churn) se produit
- Métriques (précision, rappel, AUC) standardisées
- Probabilités = $P(\text{churn})$

17. LabelEncoder ordonne **par ordre alphabétique** → 'Attrited' (A) = 0, 'Existing' (E) = 1 → classe positive = 0 ! → métriques inversées silencieusement.

Q 3.2 Encodage ordinal — Education_Level et Income_Category





18. Pas besoin de monotonie parfaite → seule la **tendance générale** compte. Les variations locales sont normales dans des données réelles.

19.

- **Encodage ordinal** : ordre défini manuellement (logique)
- **LabelEncoder** : ordre alphabétique (absurde pour 'Doctorate', 'High School', etc.)

Q 3.3 One-Hot Encoding — Card_Category (variable nominale)

```
# Q3.3 - One-Hot Encoding (Card_Category)

print("Distribution Card_Category :")
print(data['Card_Category'].value_counts())

# OHE avec drop_first pour éviter le dummy trap
ohe = pd.get_dummies(df_enc['Card_Category'], prefix='card', drop_first=True, dtype=int)
print(f"Nouvelles colonnes créées : {list(ohe.columns)}")

# Intégration dans le DataFrame
df_enc = pd.concat([df_enc.drop('Card_Category', axis=1), ohe], axis=1)
print(f"DataFrame final : {df_enc.shape[1]} colonnes")

Distribution Card_Category :
Card_Category
Blue      9436
Silver     555
Gold       116
Platinum    20
Name: count, dtype: int64
Nouvelles colonnes créées : ['card_Gold', 'card_Platinum', 'card_Silver']
DataFrame final : 21 colonnes
```

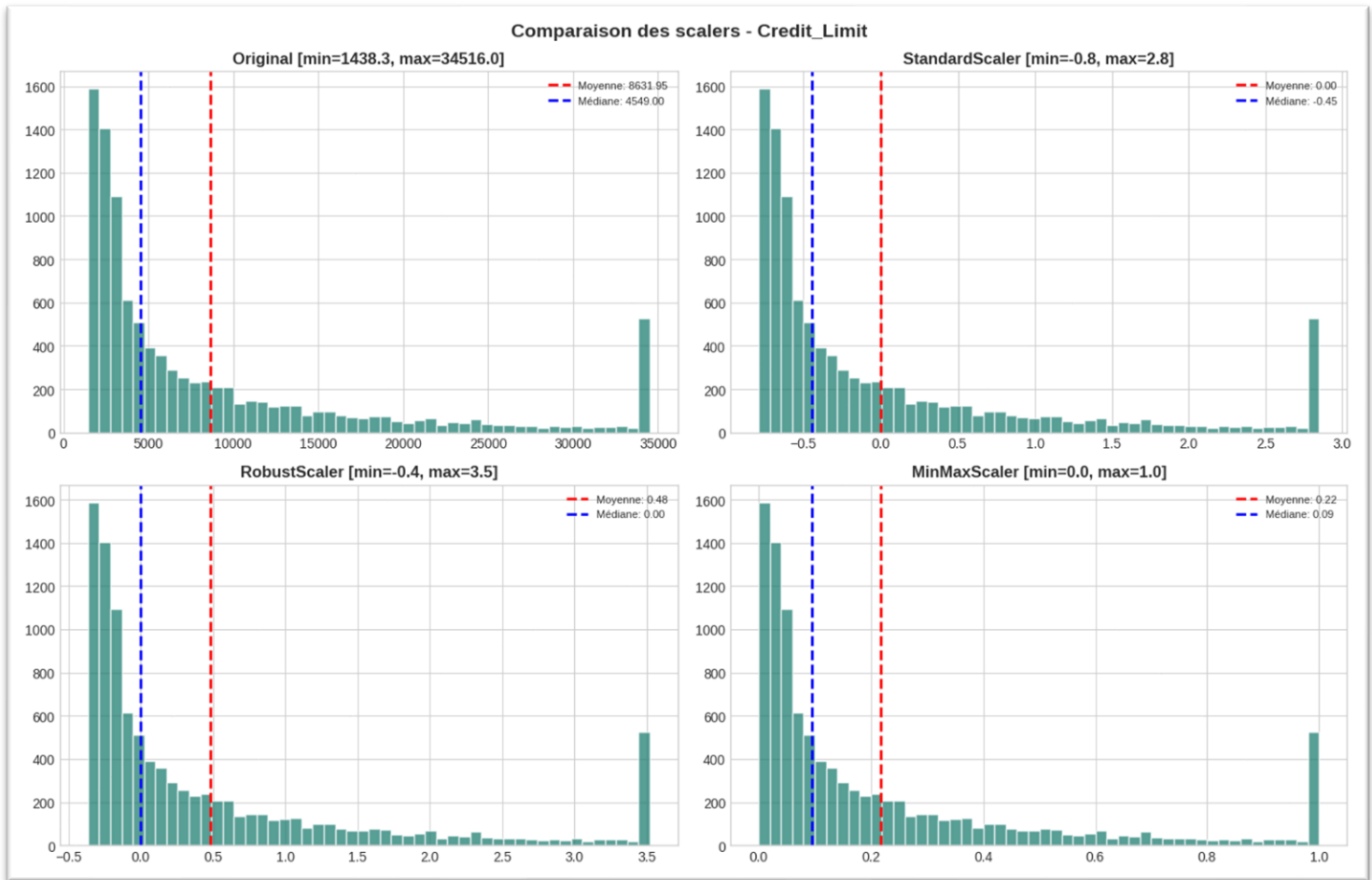
20. drop_first=True évite le **dummy trap** (multicolinéarité). On peut ne PAS l'utiliser pour les **arbres de décision** (insensibles) ou si on veut **garder toutes les colonnes**.

21. Card_Category est **nominale** (pas d'ordre Blue → Silver → Gold).

Ordinale = ordre logique (Education). **Nominale** = catégories sans hiérarchie → **OHE obligatoire** (l'encodage ordinal créerait des relations absurdes).

Partie 4 — Normalisation et Pipeline Final

Q 4.1 Comparaison visuelle des trois scalers sur Credit_Limit



22. **RobustScaler** écrase le moins les outliers — il préserve une plage plus large (min=-0,4 à max=3,5) car il utilise la médiane et l'IQR au lieu de la moyenne et l'écart-type.

23. **RobustScaler** — car les variables financières contiennent ~10% d'outliers, et RobustScaler est résistant aux valeurs extrêmes contrairement à StandardScaler (sensible) et MinMaxScaler (qui les écrase dans [0,1]).

Q 4.2 ColumnTransformer — Pipeline de preprocessing complet

24. Pour éviter le **data leakage** : `fit()` ne doit calculer les paramètres (médianes, IQR, catégories) que sur le **train set** uniquement, jamais sur le test set. Splitter avant garantit que le test set reste "invisible" pendant le preprocessing.

25.

- `fit_transform(X_train)` : calcule les paramètres sur le train set **ET** les applique
- `transform(X_test)` : applique les paramètres déjà calculés (sans les recalculer)
- **Fit de RobustScaler** : calcule la **médiane** et l'**IQR** (quartiles 25% et 75%) de chaque colonne sur le train set

26. Avec un pipeline déjà fit :

```
scaler = preprocessor.named_transformers_['num'].named_steps['scaler']
```

```
medianes = scaler.center_ # ou scaler.median_
```

Section Bonus — Pour aller plus loin

- 27.** Trans_Per_Month (corr = -0,287) n'est probablement **pas plus corrélée** que Total_Trans_Ct seule, mais c'est **intéressant** car elle normalise par la durée de relation (Months_on_book). Cela permet de comparer l'activité de clients **indépendamment de leur ancienneté** — un client récent avec peu de transactions peut être très actif (ratio élevé), tandis qu'un client ancien avec le même nombre total serait peu actif.
- 28.** Proposition : $\text{Avg_Transaction_Value} = \text{Total_Trans_Amt} / \text{Total_Trans_Ct}$ (montant moyen par transaction). Pertinent car : des clients qui churnent pourraient avoir des transactions **plus petites** (désengagement progressif) ou au contraire des **achats exceptionnels** avant de partir.